

PKCE Entropy & Brute-Force Probability, Worked Out by Hand

A secret is only as strong as its entropy. We compute the bits in a PKCE verifier, the guess probability, and contrast it with a 6-digit code that survives only because of rate limiting.

What this document does. OAuth 2.1 with PKCE defends the token exchange by binding it to a high-entropy random `code_verifier`. This document computes that entropy in bits, derives the per-guess and expected-guess attack cost, and contrasts it with a low-entropy 6-digit code — showing exactly how much work rate limiting must do when entropy is small.

Concepts covered: entropy = length $\times$ $\log_2$ (alphabet) $\cdot$ per-guess probability $\cdot$ expected guesses $2^b/2 \cdot$ rate-limited brute force $\cdot$ why short codes need throttling.

Internal learning material. Numbers reproducible from the appendix code. v1.0

1 The setup: the PKCE verifier

PKCE's `code_verifier` is a random string of $L = 43$ characters drawn from the 66-character unreserved set $\{\text{A-Z, a-z, 0-9, -, _} \}$. The client sends challenge = SHA256(verifier) up front and reveals the verifier only at token exchange, so an interceptor of the code still cannot complete the flow.

Intuition

The security of a random secret is its *entropy* — the number of equally likely values it could have been. Each character drawn from an alphabet of size A contributes $\log_2 A$ bits; a longer string from a bigger alphabet has more bits, and bits are exactly “how many guesses an attacker must try.” Everything below is counting those bits.

2 Step 1 — Entropy in bits

For a uniformly random string of length L over an alphabet of size A ,

$$\text{entropy} = L \cdot \log_2 A \text{ bits.}$$

For the PKCE verifier, $A = 66$ so $\log_2 66 = 6.044$ bits/char, and

$$b = 43 \times 6.044 = 259.9 \approx 260 \text{ bits.}$$

The search space is $2^{260} \approx 1.9 \times 10^{78}$ — comparable to the number of atoms in the observable universe.

3 Step 2 — Guess probability and expected work

Each blind guess matches with probability

$$P(\text{one guess}) = \frac{1}{2^b} = 2^{-260},$$

and the expected number of guesses to hit it (uniform search) is half the space:

$$\mathbb{E}[\text{guesses}] = \frac{2^b}{2} = 2^{259} \approx 9.3 \times 10^{77}.$$

At even 10^{12} guesses/second this is $\sim 10^{58}$ years. PKCE's verifier is brute-force-proof by a colossal margin; no rate limiting is required for the verifier itself.

Mini-example — this operation in isolation

Adding length is exponential defence. Each extra verifier character adds $\log_2 66 = 6.04$ bits, i.e. multiplies the search space by 66. Going from 43 to 44 characters multiplies attacker work by $66\times$ for one keystroke. This is why minimum-length requirements matter more than they look.

4 Step 3 — The low-entropy contrast: a 6-digit code

Now a 6-digit one-time code, $A = 10$, $L = 6$:

$$b = 6 \times \log_2 10 = 6 \times 3.32 = 19.9 \text{ bits}, \quad 2^b = 10^6 = 1,000,000.$$

Expected guesses = 5×10^5 . The defence is now entirely in the *rate*.

attacker rate r	space/ r	time to exhaust	safe?
$10^6/\text{s}$ (no limit)	$10^6/10^6$	1 s	no
100/s (loose limit)	$10^6/100$	2.8 hr	weak
5/min (strict limit)	$10^6/(5/60)$	~ 4.6 months	yes (with expiry)

The time to exhaust the space is $2^b/r$. With no throttle a 6-digit code dies in one second; a strict 5-per-minute limit plus a short expiry makes it safe. Low entropy is survivable *only* when rate limiting and expiry shrink the effective attempts.

Watch out

Entropy and rate limiting are substitutes, not luxuries. High-entropy secrets (PKCE verifiers, 256-bit tokens) need no throttle; low-entropy secrets (OTP codes, PINs) are secure *only* behind a strict attempt cap and a short TTL. A 6-digit code with no rate limit is a one-second breach — the bits are simply not there.

5 The argument, at a glance

#	Idea	Formula	Value
1	Entropy	$L \log_2 A$	260 bits (PKCE)
2	Guess probability	2^{-b}	2^{-260}
2	Expected guesses	$2^b/2$	9.3×10^{77}
3	6-digit entropy	$6 \log_2 10$	19.9 bits
3	Time to exhaust	$2^b/r$	1 s unthrottled

6 Equation cheat-sheet

Entropy (bits)	$b = L \log_2 A$
Search space	$2^b = A^L$
Per-guess probability	2^{-b}
Expected guesses	$2^b/2$
Time to exhaust	$2^b/r$ (rate r)

A Reference implementation

```
import math
def entropy_bits(L, A):
    return L * math.log2(A)

pkce = entropy_bits(43, 66)          # 259.9 bits
otp = entropy_bits(6, 10)            # 19.93 bits
print(round(pkce,1), round(otp,2))

def time_to_exhaust(bits, rate_per_s):
    return 2**bits / rate_per_s
print(time_to_exhaust(19.93, 1e6))    # 1.0 s (no limit)
print(time_to_exhaust(19.93, 5/60) / 86400) # 139 days (strict)
```

— end of document —